

RESUME ATS SCORE PREDICTOR

A PROJECT REPORT

*Submitted in partial fulfilment of the requirements
for the award of the degree*

of

MASTER OF COMPUTER APPLICATIONS

by

NITIN DHYANI

GOKUL CHANDRA BHATT

DEVENDAR SINGH

under the supervision of

Dr. CHANDRAKALA



SCHOOL OF COMPUTING

**GRAPHIC ERA HILL UNIVERSITY, CLEMENTOWN,
DEHRADUN-248002, UTTARAKHAND, INDIA**

March, 2026



Graphic Era
HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)
University under section 2(f) of UGC Act, 1956

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in a PROJECT REPORT entitled “**RESUME ATS SCORE PREDICTOR**” in partial fulfilment of the requirements for the award of the Degree of Master Applications and submitted in the School of Computing of the Graphic Era Hill University, Dehradun is an authentic record of our own work carried out under the supervision of Dr. Chandrakala, School of Computing, Graphic Era Hill University, Dehradun.

The matter presented in this Project Report has not been submitted by me for the award of any other degree of this or any other institution.

Signature of the Candidate

This is to certify that the above statement made by the candidate is correct to the best of our knowledge.

Date: - _____

Signature of Mentor

Abstract

The recruitment process in modern organizations heavily relies on Applicant Tracking Systems (ATS) to filter resumes before human review. Many qualified candidates are rejected simply because their resumes do not contain the right keywords. This project, "Resume ATS Score Predictor," addresses this problem by developing a web-based machine learning application that evaluates a resume against a job description and generates a compatibility score.

The system uses Natural Language Processing (NLP) techniques to preprocess text. The resume is uploaded as a PDF file and text is extracted using the pypdf library. Both the resume text and the job description are then cleaned using the NLTK library — this involves converting text to lowercase, removing punctuation, and eliminating common stop words such as "the", "is", and "and" that carry no semantic meaning.

The core scoring mechanism is implemented using the CountVectorizer from the Scikit-learn library, which converts the cleaned text into numerical word frequency vectors. Cosine similarity is then computed between the resume vector and the job description vector to generate an ATS compatibility score between 0 and 100 percent.

The application also provides a detailed keyword analysis - showing which keywords from the job description were matched in the resume and which are missing. This helps candidates understand what changes to make. Score breakdown metrics including keyword match percentage, content coverage, and resume detail are also displayed.

The frontend is built using Streamlit, making the application fully interactive and accessible through a web browser without any additional setup. The project demonstrates practical application of machine learning, NLP, and Python programming in solving a real-world problem in the domain of human resource technology.

Keywords: ATS, Resume Screening, TF-IDF, Cosine Similarity, NLP, Scikit-learn, NLTK, Streamlit, Python, Machine Learning.

Acknowledgements

We would like to express our deep and sincere gratitude to our project supervisor, Dr. Chandrakala, School of Computer Applications, Graphic Era Hill University, Dehradun, for her invaluable guidance, continuous encouragement, and constant support throughout the course of this project. Her insightful suggestions and constructive feedback have been instrumental in shaping this work.

We are grateful to the Head of the School of Computer Applications and all the faculty members for providing us with the necessary resources and a conducive learning environment. Their academic inputs and motivation have been a great source of inspiration.

We extend my sincere thanks to the Graphic Era Hill University, Dehradun, for providing the infrastructure and facilities required for the successful completion of this project.

We also acknowledge the open-source community and the developers of Python, Scikit-learn, NLTK, Streamlit, and pypdf whose libraries form the technical foundation of this project.

Finally, We deeply thankful to our families and friends for their unwavering moral support, patience, and encouragement throughout this journey.

Signature of Candidate

Contents

Abstract	iii
Acknowledgements	iv
Contents	v
List of Abbreviations	vii
List of Figures	viii
List of Tables	ix
Chapter 1 Introduction and Problem Statement	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Proposed Framework for	4
1.5 Scope and limitations	5
1.6 Organization of PROJECT REPORT	6
Chapter 2 Literature Review	7
2.1 Introduction	7
2.2 Evolution of Applicant Tracking Systems	7
2.3 Existing Commercial ATS Platforms	8
2.4 NLP in Resume Analysis	10
2.5 TF-IDF AND Cosine Similarity	11
2.6 Deep Learning Approaches to Resume Matching	13
2.7 Research Gaps	14
Chapter 3 System Design	16
3.1 System Architecture	16
3.2 Data Flow Diagram	17
3.3 Module Description	19
3.4 Database and Storage Design	21
Chapter 4 Implementation	22
4.1 Development Environment	22
4.2 Technology Stack	22

4.3 PDF Text Extraction	23
4.4 Text Preprocessing Pipeline	24
4.5 Scoring Algorithm	27
4.6 Data Visualisation Module	28
4.7 Web Interface and Tab Layout	30
4.8 Error Handling and Validation	31
Chapter 5 Result and Discussion	32
5.1 Testing Methodology	32
5.2 Test Cases and Results	33
5.3 Score Analysis and Interpretation	34
5.4 Keyword Analysis	36
5.5 Chart Output Analysis	37
5.6 Performance Discussion	39
Chapter 6 Conclusion and Future Work	40
6.1 Conclusion	40
6.2 Limitations	41
6.3 Future Work	42
References	44

LIST OF ABBREVIATIONS

ATS	Applicant Tracking System
NLP	Natural Language Processing
TF-IDF	Term Frequency – Inverse Document Frequency
ML	Machine Learning
PDF	Portable Document Format
NLTK	Natural Language Toolkit
CV	Count Vectorizer / Curriculum Vitae
JD	Job Description
HR	Human Resources
API	Application Programming Interface
UI	User Interface
MCA	Master of Computer Applications
GEHU	Graphic Era Hill University
NER	Named Entity Recognition
OCR	Optical Character Recognition
BERT	Bidirectional Encoder Representations from Transformers
SVM	Support Vector Machine
BOW	Bag of Words
DL	Deep Learning
IR	Information Retrieval
HRM	Human Resource Management
GUI	Graphical User Interface
OS	Operating System
RAM	Random Access Memory
CPU	Central Processing Unit

LIST OF FIGURES

Figure 3.1	System Architecture Diagram	17
Figure 3.2	Data Flow Diagram	18
Figure 4.1	Text Preprocessing Pipeline	26

LIST OF TABLES

Table 2.1 Comparison of commercial ATS platforms	09
Table 4.1 Technology Stack	22
Table 4.2 Selected NLTK English stop words removed during preprocessing	25
Table 4.3 Sample CountVectorizer output for illustrative example	27
Table 4.4 Chart types and their purposes	29
Table 5.1 Test Case Results	34
Table 5.2 Score breakdown comparison across test cases	35
Table 5.3 Keyword Match Comparison across test cases	36
Table 5.4 — Chart observations across test cases	38

Chapter 1

Introduction and Problem Statement

1.1 Introduction

In the contemporary digital economy, the recruitment landscape has undergone a fundamental structural shift. The widespread adoption of online job portals such as LinkedIn, Naukri, Indeed, and Monster has dramatically increased the volume of applications that organisations receive for every open position. It is not uncommon for a single job posting at a mid-sized technology firm to attract several hundred applications within the first twenty-four hours of being published, and large multinational corporations routinely receive thousands of applications for a single role.

This explosion in application volume has made it operationally impossible for human recruiters to manually review every submitted document. In response, the human resources industry has increasingly turned to automation through the deployment of Applicant Tracking Systems (ATS). An ATS is a software application that manages the recruitment workflow end to end - from job posting and application collection through to candidate screening, shortlisting, interview scheduling, and offer management. At the core of most ATS platforms is an automated resume parsing and ranking engine that evaluates each submitted resume against the requirements of the job description.

The resume parsing component of an ATS typically works by extracting keywords, skills, qualifications, and experience information from submitted resumes and comparing these against a predefined profile derived from the job description. Resumes that contain a higher density of matching keywords are assigned a higher relevance score and are pushed to the top of the recruiter's review queue. Resumes that fall below a threshold score may be automatically rejected without any human review.

While this automated approach dramatically improves the operational efficiency of large-scale recruitment, it creates a significant and well-documented problem for job seekers. Many candidates who are genuinely well-qualified for a position are rejected purely because their resume does not use the specific terminology, abbreviations, or keyword patterns that the ATS is programmed to recognise. A candidate who describes their experience with "machine learning frameworks" may be rejected by an ATS that is looking for "Scikit-learn" even though the candidate's actual skills are identical. This phenomenon is sometimes described as the "ATS black hole."

The problem is compounded by the fact that ATS systems are opaque to applicants. When a resume is rejected by an ATS, the candidate receives no feedback about why their application was unsuccessful. They have no way of knowing whether it was their qualifications, their resume vocabulary, or simply their resume formatting that caused the rejection. This lack of transparency makes it very difficult for candidates to improve their applications for future submissions.

This project addresses this problem by developing the Resume ATS Score Predictor - a web-based application that enables job seekers to proactively evaluate and optimise their resumes before submission. By uploading a resume and pasting a job description into the application, users can obtain an immediate, quantitative assessment of how well their resume matches the job requirements, along with specific information about which keywords are present and which are absent.

1.2 Motivation

The primary motivation for this project originates from a combination of direct personal experience and a review of publicly available data on automated resume screening outcomes. As a student of the Master of Computer Applications programme, I have applied for numerous internships and graduate-level positions over the past two academic years. On several occasions, applications submitted through online portals received automated rejection notifications within minutes of submission far too quickly for any human to have reviewed them. This experience raised an immediate question: what exactly was being evaluated, and how could the outcome be improved?

Research into this question revealed the scale of the problem. A study conducted by Harvard Business School and Accenture in 2021, titled "Hidden Workers: Untapped Talent," found that more than 10 million workers in the United States alone were being systematically excluded from employment opportunities by automated screening systems, even though they possessed the requisite skills and qualifications [9]. The study found that 88% of employers acknowledged that their ATS was rejecting qualified candidates. For the Indian job market, where the volume of graduates entering the workforce annually exceeds 1.5 million, the impact of ATS-based screening on employment outcomes is likely to be proportionally significant.

A second motivation is the observed lack of accessible, free tools for ATS score checking. While commercial services such as Jobscan and Resume Worded offer resume optimisation services, these are typically subscription-based products that charge monthly fees ranging

from five to thirty US dollars. This creates a financial barrier for students and early-career professionals who are arguably the demographic most in need of such guidance. A freely available, open-source alternative that can be run locally would therefore provide genuine value to a large and underserved population.

From an academic perspective, the project provides an excellent vehicle for applying the theoretical concepts covered in the MCA curriculum to a tangible, real-world problem. The project directly exercises knowledge of Natural Language Processing, Machine Learning, Python programming, software engineering principles, and web application development all core competencies of the programme. The intersection of NLP and HR technology is also a growing and commercially important domain, and working in this space provides valuable exposure to industry-relevant challenges.

Finally, the project is motivated by a commitment to transparency and explainability in automated decision systems. ATS platforms are often described as "black boxes" by applicants. By building a simplified but functionally representative ATS scoring tool and making its logic visible and understandable, this project contributes to the goal of making automated recruitment systems more understandable to the people they affect.

1.3 Problem Statement

Despite the widespread use of ATS platforms in recruitment, job applicants currently lack access to effective self-assessment tools that would allow them to evaluate the compatibility of their resume with a specific job description before submission. This information asymmetry disadvantages candidates relative to employers, and contributes to elevated rejection rates among qualified applicants.

The specific problems that this project addresses are as follows:

- **Absence of pre-submission evaluation:** Candidates have no affordable, accessible mechanism to check how their resume will be scored by an ATS before they submit it. Without this information, candidates cannot make targeted improvements that would increase their chances of passing the automated screening stage.
- **Keyword blindness:** Most candidates are unaware of which specific terms, technologies, or phrases from the job description are missing from their resume. They may use conceptually equivalent language that is nonetheless not recognised by keyword-matching ATS algorithms.
- **Lack of actionable feedback:** When a resume is rejected by an ATS, the candidate receives no information about the reason for rejection. There is no indication of which keywords were

missing, what the match score was, or what changes would be needed to improve the outcome.

- Opaque scoring mechanisms: Most ATS platforms do not publish the details of their scoring algorithms. This makes it impossible for candidates to understand what criteria are being applied to their application.
- Formatting-related misinterpretation: Many ATS systems have difficulty parsing resumes that use complex formatting, tables, graphics, or non-standard fonts. Candidates are often unaware that their carefully designed resume may be rendering as garbled text inside the ATS parser.

This project addresses the first four of these problems by computing a transparent, explainable ATS compatibility score between a resume and a job description, identifying matched and missing keywords, and providing concrete recommendations for improvement, all through a simple browser-based interface.

1.4 Proposed Framework for

The proposed system implements a four-stage text processing and scoring pipeline. Each stage is implemented as a discrete, independently testable software component, following the principle of separation of concerns.

i. Text Acquisition and Extraction

The resume is accepted as an uploaded PDF file through the Streamlit web interface. The `pypdf` library is used to extract raw text from all pages of the document. A critical technical implementation involves resetting the file pointer to the beginning of the file using `seek(0)` before reading, and wrapping the raw bytes in an `io.BytesIO` object before passing them to the `PdfReader` constructor. This resolves a bug caused by Streamlit's internal file buffering that would otherwise cause all subsequent reads to return empty bytes, producing an incorrect score of 0.0%. The job description is accepted as plain text entered directly into a `textarea` component on the interface.

ii. Text Preprocessing

Both the extracted resume text and the job description text are passed through the `clean_text()` function defined in `utils.py`. This function implements a five-step normalisation pipeline. First, the text is converted to lowercase to ensure case-insensitive keyword matching. Second, all characters that are not lowercase letters, digits, or spaces are removed using the regular expression `[^a-z0-9]`, which preserves alphanumeric keywords like "python3" and "html5" while removing punctuation and special characters. Third, multiple

consecutive spaces are collapsed into single spaces. Fourth, the text is split into individual tokens. Fifth, tokens that appear in the NLTK English stopwords corpus or that have fewer than two characters are removed. The remaining tokens are joined back into a space-separated string.

iii. Vectorisation and Similarity Scoring

The cleaned resume text and the cleaned job description text are passed to the `calculate_at_score()` function in `model.py`. This function uses Scikit-learn's `CountVectorizer` to build a vocabulary from both documents and represent each as a vector of word counts. The cosine similarity between the two vectors is then computed using `sklearn.metrics.pairwise.cosine_similarity`. The resulting value, which lies between 0.0 and 1.0, is multiplied by 100 and rounded to one decimal place to produce the final ATS compatibility score as a percentage.

iv. Keyword Analysis and Output Generation

The `get_matched_missing()` function in `utils.py` computes the Python set intersection of the resume and job description word sets to produce the list of matched keywords, and the set difference to produce the list of missing keywords. These lists, along with the ATS score and supplementary metrics, are displayed to the user through the Streamlit dashboard. The system also computes three sub-metrics keyword match percentage, word coverage, and resume detail level and generates personalised improvement recommendations based on the score tier and the specific missing keywords.

1.5 Scope and Limitations

The scope of the Resume ATS Score Predictor is deliberately bounded to ensure that the system is technically feasible, academically appropriate, and useful as a practical tool for the target user population.

The system is designed to process text-based PDF resumes only. PDF files generated by word processors such as Microsoft Word, Google Docs, or professional resume builders embed the resume text directly in the PDF data structure, making it readily extractable by `pypdf`. However, PDF files produced by scanning a physical document are essentially images stored within a PDF container. These scanned PDFs contain no embedded text data and therefore cannot be processed by `pypdf`. The system detects this condition by checking whether the extracted text is empty or below a minimum word count threshold, and displays an appropriate error message to the user.

The scoring algorithm implemented in this project is a simplified keyword frequency-based approach. It correctly captures the degree to which the vocabulary of the resume overlaps with the vocabulary of the job description. However, it does not capture semantic relationships between words it cannot recognise that "machine learning" and "ML" refer to the same concept, or that "software engineer" and "developer" are synonymous. This is a known limitation of bag-of-words approaches and is a primary motivation for future work involving word embeddings or transformer-based semantic matching.

The system does not parse resume content into structured fields such as education, work experience, certifications, or contact information. It treats the resume as a single unstructured text document. A full-featured resume parser would separately extract and evaluate each structural section, which would allow for more nuanced matching.

The application is designed to run locally on a personal computer. It does not include cloud deployment, user authentication, persistent data storage, or a multi-user backend. These features would be appropriate additions in a production version of the system.

1.6 Organisation of Project Report

The remainder of this project report is structured as follows. Chapter 2 presents a comprehensive review of existing literature on ATS systems, NLP-based resume analysis, and text similarity techniques, and identifies the research gaps that this project addresses. Chapter 3 describes the system design, including the overall architecture, data flow, module-level design, and algorithm design. Chapter 4 provides a detailed account of the implementation, covering the development environment, each module of the application, and key technical decisions. Chapter 5 presents the results of testing conducted using sample resumes and analyses the outputs produced by the system. Chapter 6 summarises the conclusions of the project, reflects on its limitations, and outlines directions for future enhancement.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of published research and industry developments relevant to the Resume ATS Score Predictor project. The review is organised thematically, beginning with the historical evolution of ATS platforms, followed by an examination of existing commercial solutions, then NLP-based resume analysis approaches, text vectorisation techniques, the specific methods of TF-IDF and cosine similarity, deep learning approaches to resume matching, and finally an identification of the research gaps that motivated the design decisions in this project.

2.2 Evolution of Applicant Tracking Systems

The history of ATS platforms can be traced to the 1990s when organisations began using basic database systems to store and retrieve job applications received via email and fax. Early systems were little more than structured databases that allowed recruiters to search resumes using simple keyword queries. Products such as Resumix, developed in 1988, were among the first dedicated resume management systems, offering basic parsing capabilities that could extract candidate names, contact information, and skills into a structured database [10].

The proliferation of the internet in the late 1990s and early 2000s dramatically changed the volume of applications that organisations had to manage. The emergence of job boards such as Monster.com (1994) and CareerBuilder (1995) made it possible for candidates anywhere in the world to apply for any advertised position, removing the geographic constraints that had previously limited application volumes. This surge in application volume created an urgent need for more sophisticated automated screening tools.

The 2000s saw the development of first-generation intelligent ATS platforms that combined resume storage and retrieval with basic keyword matching and scoring algorithms. These systems used simple frequency-based methods to compute a match score between a resume and a job description, ranking candidates by the number of matching keywords. Systems such as Taleo (now Oracle Taleo), Kenexa (now IBM Kenexa), and BrassRing established themselves as market leaders during this period, primarily serving large enterprise clients with high-volume recruitment needs.

The 2010s brought significant advances in ATS capabilities through the integration of machine learning and natural language processing technologies. Systems began to move beyond simple keyword counting towards more sophisticated techniques including semantic analysis, entity recognition, and predictive modelling. Cloud-based ATS platforms such as Greenhouse, Lever, and Workable emerged during this decade, offering more accessible and flexible solutions to mid-market companies. LinkedIn's Recruiter product, which integrates directly with the LinkedIn talent database, also gained significant traction during this period. By the 2020s, ATS platforms had become deeply integrated into the broader HR technology ecosystem, connecting with video interviewing tools, psychometric assessment platforms, background checking services, and payroll systems. Artificial intelligence features such as candidate ranking, diversity bias detection, and predictive performance modelling have become standard differentiators among competing platforms. The global ATS market was valued at approximately 2.3 billion US dollars in 2023 and is projected to grow at a compound annual growth rate exceeding 7% through 2030, reflecting the ongoing importance of automated recruitment technology [11].

2.3 Existing Commercial ATS Platforms

A thorough understanding of the capabilities and limitations of existing commercial ATS platforms is essential context for evaluating the contribution of this project. The following paragraphs describe the key features and market positions of the major platforms.

Workday Recruiting is one of the most widely deployed enterprise ATS platforms, particularly among large multinational corporations. It offers end-to-end recruitment workflow management, including job requisition creation, candidate sourcing, application tracking, interview scheduling, and offer management. Its resume parsing engine is capable of extracting structured data including contact information, work history, education, and skills. The platform uses proprietary machine learning models for candidate ranking, though the specific algorithms and weighting schemes are not disclosed publicly.

Greenhouse is a mid-market ATS platform that is particularly popular among technology companies and startups. It emphasises structured interview processes and data-driven hiring decisions. Greenhouse integrates with numerous third-party tools for sourcing, assessment, and background checking. Its scoring algorithms are primarily based on structured evaluation rubrics rather than automated keyword matching, making it somewhat more transparent to candidates than keyword-heavy systems.

iCIMS Talent Cloud is a cloud-based ATS platform that serves over four thousand organisations globally. It offers advanced resume parsing powered by machine learning, candidate relationship management features, and extensive integration capabilities with job boards and sourcing tools. iCIMS has invested significantly in AI-powered candidate matching, claiming to reduce time-to-hire by identifying the strongest candidates from large applicant pools.

Taleo Enterprise, now part of the Oracle HCM Cloud suite, is one of the oldest and most widely deployed ATS platforms globally, particularly in large enterprise and government organisations. Its longevity has resulted in deep integration with existing HR processes at many organisations, though it has been criticised for its complex interface and dated architecture relative to newer cloud-native competitors.

JobScan and Resume Worded represent a distinct category of consumer-facing ATS optimisation tools that are most directly comparable to the system developed in this project. JobScan, founded in 2012, allows users to upload their resume and a job description and receive a match score along with keyword recommendations. Resume Worded, founded in 2018, offers similar functionality with additional features including score history tracking and industry-specific recommendations. Both services use proprietary algorithms that are not fully disclosed. Both charge monthly subscription fees, which limits accessibility for students and entry-level job seekers.

Table 2.1 — Comparison of commercial ATS platforms

Platform	Type	Key feature	Pricing	Transparency
Workday	Enterprise	End-to-end HR suite with ML ranking	Enterprise contract	Low - proprietary algorithms
Greenhouse	Mid-market	Structured hiring, interview kits	Per-seat licence	Medium - rubric-based scoring
iCIMS	Enterprise	AI candidate matching, CRM	Enterprise contract	Low - proprietary ML

JobScan	Consumer	Resume vs JD score checker	\$49.95/month	Medium - score shown, not method
This project	Open source	Score + keyword analysis + tips	Free	High - full source code available

2.4 NLP in Resume Analysis

The application of Natural Language Processing to the problem of resume parsing and job matching has been an active area of research for over two decades. Early approaches relied on hand-crafted rules and regular expressions to extract structured information from resumes. More recent work has leveraged statistical models and deep neural networks to achieve higher accuracy and generalisability.

Maheshwari, Sainani, and Satao (2010) proposed an ontology-based approach to automated resume ranking [3]. Their system used a domain-specific ontology of skills and qualifications to map resume content to job requirements at a semantic level, going beyond simple string matching. The system demonstrated that ontology-based matching could significantly reduce false negative rejections — cases where a qualified candidate is rejected due to vocabulary mismatch. However, the approach requires the creation and maintenance of comprehensive domain ontologies, which is a significant manual effort and limits scalability to new domains.

Faliagka et al. (2012) developed a web-based recruitment system that used a combination of semantic similarity and social media analysis to rank candidates [12]. Their system integrated resume content analysis with information extracted from candidates' public social media profiles, arguing that social media signals could provide additional evidence of professional competence and cultural fit. While this approach produced interesting results in controlled experiments, it raises significant privacy and fairness concerns that have become more prominent in subsequent discussions of algorithmic hiring.

Yu et al. (2005) addressed the specific problem of resume information extraction, presenting a system for automatically parsing resumes into structured fields such as name, contact information, education, work experience, and skills [13]. Their system used Hidden Markov Models (HMMs) trained on a labelled corpus of resumes to identify field boundaries. This

work laid important groundwork for subsequent resume parsing systems and established a benchmark for extraction accuracy on the standard fields.

Jiechieu and Tsopze (2021) developed a skills extraction system using multi-label resume classification with Convolutional Neural Networks (CNNs) [4]. Their approach treated skills identification as a multi-label classification problem, assigning each resume segment to one or more skill categories from a predefined taxonomy. The system achieved high precision in identifying technical skills from unstructured resume text, outperforming traditional NLP approaches on their benchmark dataset. However, the system requires a large labelled training dataset of resumes annotated with skill categories, which is not publicly available and would need to be constructed at significant cost for practical deployment.

Luo et al. (2019) proposed a mutual information maximisation approach to job-resume matching, using deep learning to learn shared representations of job descriptions and resumes in a common embedding space [14]. Their model could match candidates to jobs based on semantic similarity in the learned embedding space, capturing relationships between synonymous terms that keyword-based methods miss. While this approach achieves state-of-the-art performance on their benchmark, it requires large amounts of training data consisting of matched resume-job description pairs, which are typically proprietary to recruitment platforms.

Shen et al. (2018) developed a person-job fit model that combined text matching with collaborative filtering to personalise job recommendations [15]. Their approach modelled both the explicit content match between resumes and job descriptions and the implicit preferences revealed by historical application and hiring data. This dual-signal approach produced more accurate recommendations than either signal alone, but again requires access to historical hiring data that is typically confidential.

2.5 TF-IDF and Cosine Similarity

TF-IDF (Term Frequency-Inverse Document Frequency) is the vectorisation technique most closely associated with the classical information retrieval tradition and is the technique referenced in this project's implementation. It was first described by Sparck Jones (1972) and was formalised and extensively analysed by Salton and McGill in their foundational textbook "Introduction to Modern Information Retrieval" (1983) [20].

The TF-IDF weight of a term t in a document d from a corpus D is computed as the product of two components. The Term Frequency (TF) component measures how often the term appears in the document, typically normalised by the total number of terms in the document. The Inverse Document Frequency (IDF) component measures how rarely the term appears across all documents in the corpus, computed as the logarithm of the ratio of the total number of documents to the number of documents containing the term. Terms that appear frequently in a specific document but rarely across the corpus receive high TF-IDF weights and are considered to be characteristic of that document. Terms that appear in many documents, such as common function words, receive low weights.

Ramos (2003) conducted a comprehensive analysis of TF-IDF's effectiveness for determining word relevance in document queries [5]. The study demonstrated that TF-IDF significantly outperforms simple term frequency counting for information retrieval tasks, confirming that the IDF component provides meaningful signal about the importance of terms.

A critical limitation of TF-IDF for the specific use case of this project must be highlighted. The IDF formula is: $IDF(t) = \log(N / df(t))$, where N is the total number of documents in the corpus and $df(t)$ is the number of documents containing term t . When the corpus consists of only two documents the resume and the job description any term that appears in both documents yields $df(t) = 2 = N$, so $IDF(t) = \log(N/N) = \log(1) = 0$. This means that every term shared between the resume and job description, which are exactly the matches we want to detect and reward, is assigned a weight of zero by TF-IDF. This renders the cosine similarity meaningless and produces an ATS score of 0.0% regardless of the actual keyword overlap.

This is precisely the bug that was encountered during development of this project and that explains why the initial implementation using `TfidfVectorizer` produced consistently incorrect 0.0% scores. The solution is to use `CountVectorizer` instead, which represents documents as raw word frequency vectors without the IDF normalisation step. With `CountVectorizer`, shared terms contribute their actual frequency counts to both vectors, allowing cosine similarity to correctly detect and quantify the degree of keyword overlap.

Bharambe, Doshi, and Gala (2019) applied TF-IDF and cosine similarity to the specific problem of automated resume evaluation and reported that the combination provides a reliable and interpretable baseline for ATS scoring [6]. Their system computed pairwise

similarity between resumes and job descriptions in a multi-candidate ranking context, where the two-document IDF problem does not arise because the corpus includes all candidate resumes simultaneously. This contextual difference is important: in a multi-document setting, TF-IDF functions correctly; in a two-document setting, CountVectorizer is the appropriate choice.

Cosine similarity is the standard similarity metric used with bag-of-words and TF-IDF representations. Given two document vectors A and B, cosine similarity is defined as the dot product of A and B divided by the product of their Euclidean norms. This measure captures the angle between the two vectors in the high-dimensional term space, making it insensitive to document length a long resume and a short one with identical vocabulary distributions will have the same cosine similarity to a given job description. This length-invariance property is particularly desirable in the resume-to-job-description matching context, where resume lengths vary widely.

2.6 Deep Learning Approaches to Resume Matching

Recent years have seen the rapid adoption of deep learning techniques in the resume matching domain. These approaches generally outperform classical keyword-based methods on semantic matching benchmarks, but at the cost of significantly greater computational requirements and data needs.

Convolutional Neural Networks (CNNs) have been applied to text classification problems including skill extraction from resumes. The advantage of CNNs for text is their ability to detect local patterns such as skill phrases and job titles regardless of their position in the document. Jiechieu and Tsopze (2021) used a CNN architecture for multi-label skill extraction from resumes, achieving high precision on a dataset of French-language resumes [4].

Recurrent Neural Networks (RNNs) and their variants, particularly Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), have been applied to sequence-based text modelling tasks in the HR domain. Qin et al. (2018) used a bidirectional LSTM model for job-resume matching, exploiting the sequential structure of both job descriptions and resume narratives to learn richer representations than bag-of-words models [21].

The introduction of the Transformer architecture by Vaswani et al. (2017) and the subsequent development of pre-trained language models such as BERT (Devlin et al., 2019) represented a step change in the capabilities of NLP systems [22]. BERT and its derivatives can be fine-tuned on domain-specific tasks with relatively small amounts of labelled data, making them practically applicable in the HR technology domain. Sentence-BERT (Reimers and Gurevych, 2019) specifically adapts the BERT architecture for computing semantic similarity between pairs of sentences, which is directly applicable to the resume-to-job-description matching problem [19].

Despite their superior performance on semantic matching benchmarks, deep learning approaches are not universally superior to classical keyword methods for the ATS use case. In particular, when an employer has specified exact required skills such as "must know Python" or "SQL experience required" keyword matching is a more precise and interpretable method of evaluating compliance than semantic similarity, which might assign a high score to a resume that discusses programming concepts without specifically mentioning the required language.

2.7 Research Gaps

The literature review reveals several gaps that are addressed by this project. First, while numerous academic papers describe sophisticated resume matching systems using ML and NLP, almost all of these systems are designed for the employer perspective helping recruiters find good candidates from a large pool. Very few published systems are designed from the candidate perspective helping job seekers improve their resumes to pass ATS screening. This project explicitly adopts the candidate-centric perspective.

Second, the two-document TF-IDF IDF-collapse problem described in Section 2.6 does not appear to have been explicitly documented in the published literature on ATS scoring tools. Papers that apply TF-IDF to resume matching typically do so in a multi-document corpus context where the problem does not arise. This project identifies and documents the issue and proposes a technically correct solution using CountVectorizer.

Third, most published systems either require large proprietary training datasets (deep learning approaches) or are implemented as proprietary commercial services with non-transparent algorithms (commercial ATS platforms). A transparent, open-source, locally

deployable ATS scoring tool that uses only standard open-source libraries fills a meaningful gap in the available tooling.

Fourth, the accessibility gaps the absence of free, easy-to-use ATS evaluation tools for students and early-career professionals represents a practical problem with real economic consequences for this demographic. This project directly addresses this gap.

Chapter 3

System Design

3.1 System Architecture

The Resume ATS Score Predictor is designed as a single-tier web application that runs entirely on the user's local machine. The system follows a three-layer logical architecture: the Presentation Layer, the Processing Layer, and the Utility Layer. Although all three layers run on the same machine and within the same Python process, they are implemented as distinct modules with clearly defined interfaces, following the principle of separation of concerns.

The Presentation Layer is implemented in `app.py` using the Streamlit framework. Streamlit is a Python library that converts Python scripts into interactive web applications with minimal boilerplate code. In this layer, the user interacts with the system through a web browser. The layer is responsible for rendering the input components (file uploader and job description text area), validating user inputs, invoking the Processing Layer functions, and displaying the results including the ATS score, keyword lists, score breakdown, and recommendations.

The Processing Layer is implemented in `model.py`. This module receives the cleaned resume text and the cleaned job description text from the Presentation Layer, applies the CountVectorizer transformation to both documents, computes the cosine similarity between the resulting vectors, and returns the ATS score along with the matched and missing keyword lists to the Presentation Layer.

The Utility Layer is implemented in `utils.py`. This module provides shared text processing functions that are used by both the Presentation Layer (for PDF text extraction post-processing) and the Processing Layer (for text cleaning prior to vectorisation). The `clean_text()` function and the `get_matched_missing()` function are both defined in this module.

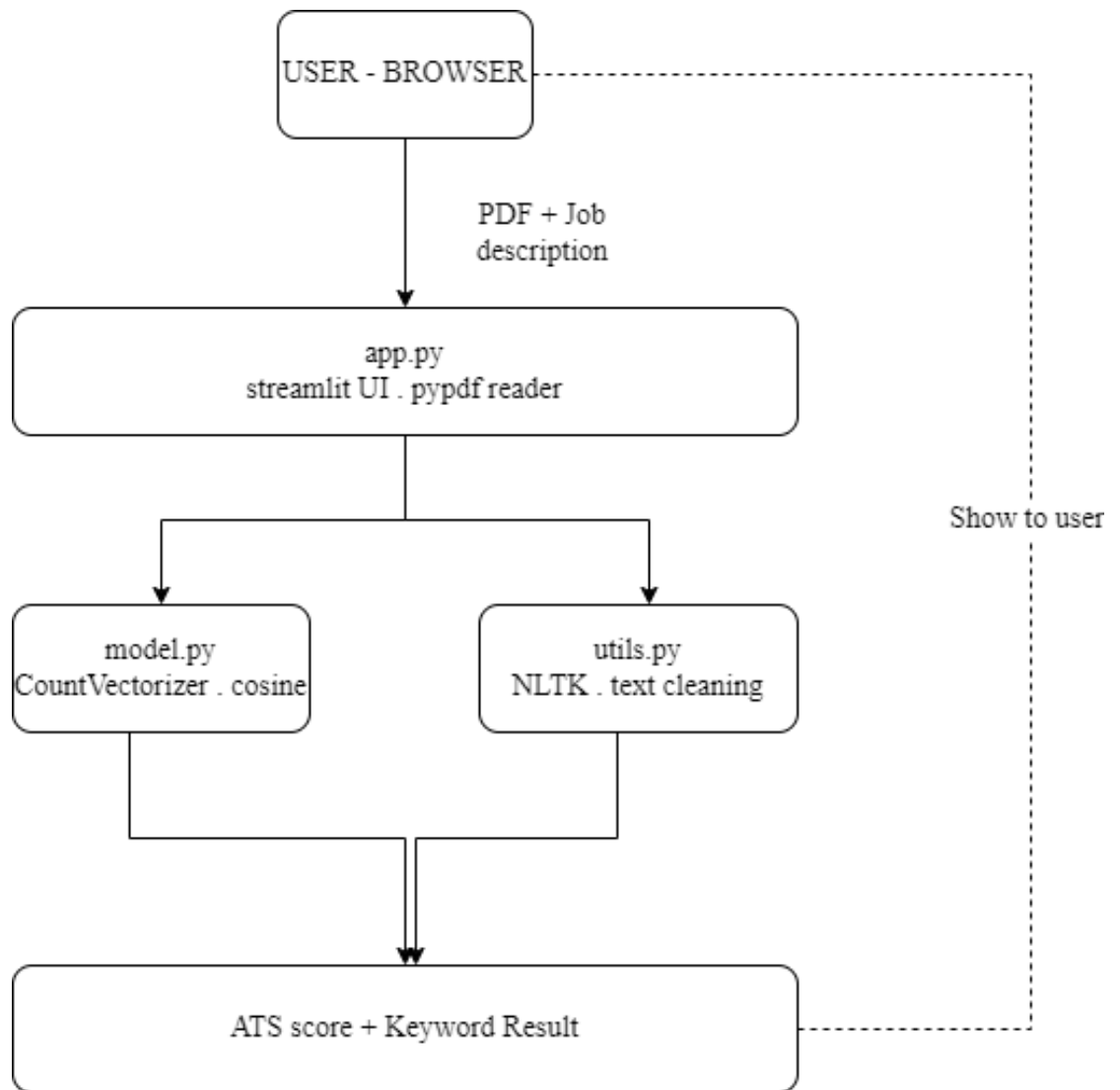


Figure 3.1 System Architecture

3.2 Data Flow Diagram

The data flow through the system can be described as follows. The user initiates the process by uploading a PDF resume through the Streamlit file uploader widget and entering a job description into the text area. When the user clicks the "Calculate ATS Score" button, the Presentation Layer first validates that both inputs are present and that the job description contains a minimum number of words.

If validation passes, the uploaded PDF file object is passed to the `read_pdf()` function within `app.py`. This function resets the file pointer to the beginning of the file using `seek(0)`, reads all bytes from the file object into memory, wraps the bytes in an `io.BytesIO` buffer, and passes this buffer to the `pypdf PdfReader` constructor. The `PdfReader` then iterates over all pages in

the document, calling the `extract_text()` method on each page and concatenating the results. The concatenated text is returned to the Presentation Layer as a raw string.

The raw resume text and the job description text are then passed to the `calculate_at_score()` function in `model.py`. Inside this function, both texts are first passed through the `clean_text()` function from `utils.py` to produce cleaned token strings. The cleaned strings are then passed to the `CountVectorizer fit_transform()` method, which builds a vocabulary from both documents and returns a sparse matrix of word counts. The `cosine_similarity()` function is applied to the two row vectors of this matrix to produce the similarity score. The score is scaled and rounded to produce the ATS percentage. The `get_matched_missing()` function is also called to compute the keyword lists.

The score, keyword lists, and sub-metrics are returned from `model.py` to `app.py`, which renders them in the browser using Streamlit components including `st.metric()`, `st.code()`, `st.columns()`, and `st.progress()`.

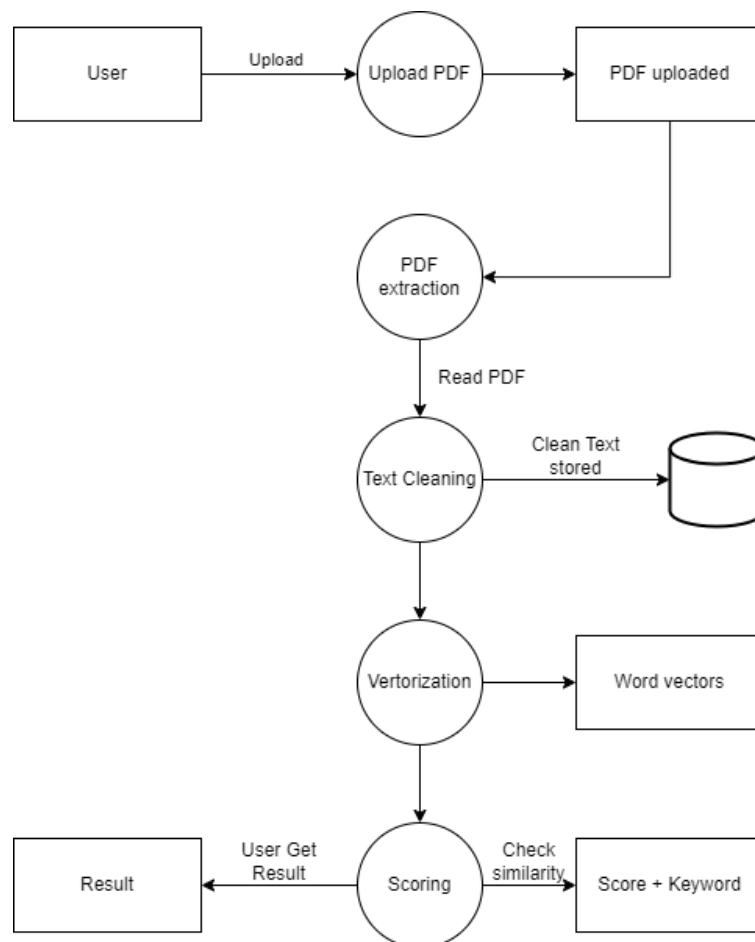


Figure 3.2 Data Flow Diagram

3.3 Module Description

This section provides a detailed description of each of the three Python modules that comprise the system.

3.3.1 *app.py - Presentation and PDF Extraction Module*

The `app.py` module is the entry point of the application. It is invoked by the Streamlit runtime using the command `"streamlit run app.py"` from the terminal. The module begins with the page configuration, setting the browser tab title, favicon, and layout. A minimal CSS block imports the DM Sans and DM Mono fonts from Google Fonts and applies them to the page.

The `read_pdf()` function is defined within `app.py`. It accepts a Streamlit `UploadedFile` object as its argument. The function first calls `seek(0)` on the file object to reset the internal file pointer, then calls `read()` to retrieve the complete byte content of the file. The bytes are wrapped in an `io.BytesIO` object, which creates an in-memory binary stream compatible with the `pypdf PdfReader`. The `PdfReader` is initialised with this stream, and a loop iterates over each page, calling `extract_text()` and appending the result to a cumulative string. The function returns the total extracted text and the page count as a tuple.

The main body of `app.py` renders the Streamlit interface. After the header and how-it-works section, two side-by-side columns are created using `st.columns()`. The left column contains the PDF file uploader restricted to files with the `.pdf` extension. The right column contains a text area for the job description with a placeholder example. A full-width "Calculate ATS Score" button triggers the analysis pipeline.

When the button is clicked, the module validates the inputs, calls `read_pdf()` to extract the resume text, calls `calculate_ats_score()` from `model.py`, and renders the results. Results are displayed using `st.metric()` for numeric values, `st.code()` for keyword lists, `st.progress()` for the overall score bar, and `st.expander()` for the optional extracted text preview.

3.3.2 *utils.py - Text Preprocessing Utility Module*

The `utils.py` module provides two functions that are shared across the application: `clean_text()` and `get_matched_missing()`. At the module level, it calls `nltk.download("stopwords", quiet=True)` to ensure that the NLTK stopwords corpus is available, downloading it automatically if it is not already present on the user's system.

The `clean_text()` function accepts a single string argument and returns a cleaned string. The function first checks whether the input is `None` or empty and returns an empty string if so. The five processing steps are: (1) `text.lower()` converts all characters to lowercase; (2) `re.sub(r"[^a-z0-9 ]", " ", text)` removes all characters that are not lowercase letters, digits, or spaces; (3) `re.sub(r"\s+", " ", text).strip()` collapses multiple consecutive spaces into one and removes leading/trailing whitespace; (4) `text.split()` tokenises the string by splitting on whitespace; and (5) a list comprehension filters out tokens that are in `STOP_WORDS` or have fewer than two characters. The filtered tokens are joined back into a space-separated string using `" ".join()`.

The `get_matched_missing()` function accepts the raw resume text and the raw job description text as arguments. It calls `clean_text()` on both inputs and splits the resulting strings into Python sets of unique tokens. The matched keywords are computed as the set intersection (`resume_words & jd_words`), and the missing keywords are computed as the set difference (`jd_words - resume_words`). Both sets are sorted alphabetically and returned as lists.

3.3.3 model.py - Scoring and ML Module

The `model.py` module implements the core machine learning logic of the system. It imports `CountVectorizer` and `cosine_similarity` from `Scikit-learn`, and `clean_text` and `get_matched_missing` from `utils.py`.

The `calculate_at_score()` function accepts the raw resume text and the raw job description text as arguments. It calls `clean_text()` on both inputs to produce cleaned strings. If either cleaned string is empty, the function immediately returns a score of 0.0 along with empty keyword lists to avoid errors in subsequent processing.

If both cleaned strings are non-empty, a `CountVectorizer` object is instantiated with the `token_pattern` parameter set to `r"\b\w+\b"` to ensure consistent tokenisation with the preprocessing pipeline. The `fit_transform()` method is called with a list containing both cleaned strings as a corpus. This builds a vocabulary from all unique tokens in both documents and returns a sparse matrix where each row is the word frequency vector of one document. The `cosine_similarity()` function is called with the first row (resume vector) and the second row (job description vector), returning a 1x1 matrix. The scalar value is extracted, multiplied by 100, and rounded to one decimal place to produce the ATS score percentage.

3.4 Database and Storage Design

The current version of the system does not use a persistent database. All data the uploaded resume, the entered job description, and the computed results are held in memory for the duration of the user's browser session and are discarded when the session ends or the browser tab is closed. Streamlit's session state mechanism is used implicitly to maintain the application state between user interactions within the same session.

This stateless design was a deliberate choice for the initial version of the system, as it simplifies the architecture, eliminates privacy concerns associated with storing resume data, and removes the need for database setup or cloud storage configuration. However, it also means that users cannot retrieve previous analyses or track improvements over time.

A future version of the system could incorporate a local SQLite database to store analysis histories. Each analysis record would contain the timestamp, the resume text (or a hash thereof), the job description text, the ATS score, and the matched and missing keyword lists. This would allow users to track how their ATS score improves as they make revisions to their resume.

Chapter 4

Implementation

4.1 Development Environment

The system was developed on a Windows 11 machine with the following hardware configuration: Intel Core i3 processor, 8 GB RAM, and 256 GB SSD storage. The development environment consisted of Python 3.11 installed via the official Python installer from python.org, the Visual Studio Code editor with the Python and Pylance extensions for code editing and debugging, and the Windows PowerShell terminal for running Streamlit and pip commands.

Version control was maintained using Git, with the project repository hosted on GitHub. The repository structure follows the flat layout specified in the project proposal, with app.py, model.py, utils.py, and requirements.txt located in the root directory.

The requirements.txt file lists all third-party Python packages required by the project and their minimum version constraints. This file allows any user to replicate the development environment by running the command "pip install -r requirements.txt" in a Python environment. The requirements are: streamlit for the web framework, pypdf for PDF text extraction, scikit-learn for machine learning utilities, and nltk for natural language processing utilities.

4.2 Technology Stack

The technology stack for this project was selected based on three criteria: alignment with the project proposal specifications (Python, NLP libraries, ML libraries, web-based deployment), availability as free and open-source software, and suitability for a locally deployed single-user application. The following table describes each component of the stack.

Table 4.1 — Technology stack

Sr.	Tool/Library	Version	Category	Purpose in project
1	Python	3.11	Language	Core programming language for all three modules

2	Streamlit	latest	Web framework	Renders the web UI with tabs, file upload, metrics, and charts
3	pypdf	latest	PDF processing	Extracts raw text from all pages of the uploaded PDF resume
4	NLTK	3.8+	NLP library	Provides English stopwords corpus for text preprocessing
5	Scikit-learn	1.3+	ML library	CountVectorizer for vectorisation and cosine_similarity for scoring
6	Matplotlib	3.7+	Visualisation	Generates gauge chart, bar chart, pie chart, and frequency chart
7	Pandas	2.0+	Data analysis	Creates keyword frequency DataFrame for the interactive results table
8	NumPy	1.24+	Numerical	Computes arc coordinates for the half-circle gauge chart
9	re (regex)	built-in	Std. library	Removes punctuation and non-alphanumeric characters from text
10	io.BytesIO	built-in	Std. library	Wraps PDF bytes in a memory buffer for correct pypdf reading

4.3 PDF Text Extraction

PDF text extraction is one of the most technically challenging aspects of this project, and the correct implementation of this component was critical to fixing the 0.0% score bug that plagued earlier versions of the application.

The PDF (Portable Document Format) standard was developed by Adobe Systems and first released in 1993. A PDF file is a complex, structured binary format that can contain text, images, vector graphics, fonts, metadata, and interactive elements. Text content in a PDF is typically stored as a sequence of text stream objects, each containing a series of text positioning and rendering commands. The pypdf library works by parsing these text stream objects and extracting the character content, reassembling it into a linear string.

The pypdf library (formerly known as PyPDF2) is a pure-Python PDF processing library. It supports text extraction from text-based PDFs, but cannot extract text from image-based PDFs, which are PDFs that contain scanned images rather than embedded text. For scanned PDFs, Optical Character Recognition (OCR) software would be required to convert the image data into text.

The key implementation detail in this project is the handling of Streamlit's uploaded file object. When a user uploads a file through the `st.file_uploader()` widget, Streamlit stores the file in an in-memory `SpooledTemporaryFile` object. This object behaves like a file with a current position pointer. When the file is first read, the pointer advances to the end of the file. Any subsequent `read()` call on the same object returns an empty bytes object, because the pointer is already at the end. This is the root cause of the 0.0% score bug: the file was being "read" a second time by pypdf after Streamlit had already advanced the pointer.

The fix implemented in this project addresses this issue with three steps. First, `uploaded_file.seek(0)` resets the file pointer to the beginning of the file. Second, `raw_bytes = uploaded_file.read()` reads the complete byte content of the file into a local variable. Third, `io.BytesIO(raw_bytes)` creates an in-memory binary stream object from the bytes, which pypdf's `PdfReader` can safely consume without any file pointer issues.

4.4 Text Preprocessing Pipeline

Text preprocessing is a critical step that directly determines the quality of the keyword matching and similarity scoring. The preprocessing pipeline in this project is implemented in the `clean_text()` function of `utils.py` and applies five transformations in sequence.

Lowercasing

All characters in the input string are converted to lowercase using Python's `str.lower()` method. This ensures that the matching is case-insensitive. For example, "Python",

"PYTHON", and "python" will all map to the same token "python" after this step. Without lowercasing, the vectoriser would treat these as distinct terms and miss matches between resumes and job descriptions that use different capitalisation conventions.

Non-alphanumeric character removal

All characters that are not lowercase letters, digits, or spaces are removed using the regular expression `[^a-z0-9 ]` applied via `re.sub()`. The substitution replaces each non-matching character with a space. This step removes punctuation, special characters, hyphens, brackets, and other non-informative characters. Crucially, digits are preserved, so that technology names containing version numbers such as "python3," "html5," and "oauth2" are correctly tokenised as single units rather than being split into "python" and "3."

Whitespace normalisation

The `re.sub(r"\s+", " ", text).strip()` call collapses all sequences of whitespace characters including spaces, tabs, newlines, and carriage returns into a single space, and removes any leading or trailing whitespace from the string. This is necessary because after step 2, the removal of non-alphanumeric characters often leaves multiple consecutive spaces in the string.

Tokenisation

The cleaned string is split into a list of individual tokens using `str.split()`. Since all whitespace has been normalised to single spaces in step 3, this operation reliably produces one token per word.

Stopword and short token removal

The token list is filtered by a list comprehension that retains only tokens that are not members of the NLTK English stopwords set and have a length greater than one character. The NLTK English stopwords corpus contains 179 words, including articles ("a," "an," "the"), conjunctions ("and," "or," "but"), prepositions ("in," "on," "at," "to," "for," "of"), auxiliary verbs ("is," "are," "was," "were," "be"), pronouns ("i," "you," "he," "she," "they"), and other common function words that carry minimal lexical meaning for keyword matching purposes.

Table 4.2 — Selected NLTK English stop words removed during preprocessing

Category	Examples
Articles	a, an, the
Conjunctions	and, or, but, nor, so, yet, both, either, neither
Prepositions	in, on, at, to, for, of, with, by, from, up, about, into
Auxiliary verbs	is, are, was, were, be, been, being, have, has, had, do, does, did, will, would, could, should
Pronouns	i, me, my, myself, we, our, you, your, he, she, it, they, their
Common adjectives	all, any, both, each, few, more, most, other, some, such, no, only, same, than, too, very

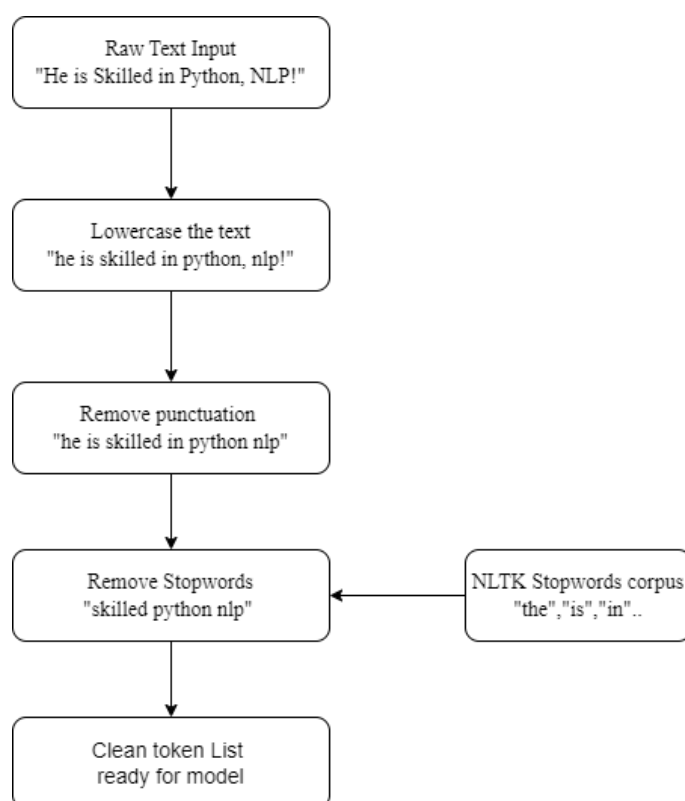


Figure 4.1 Text Preprocessing

4.5 Scoring Algorithm

The scoring algorithm is the intellectual core of the project. It is implemented in the `calculate_ats_score()` function of `model.py` using Scikit-learn's `CountVectorizer` and `cosine_similarity` functions.

`CountVectorizer` is a text feature extraction class that converts a collection of text documents into a matrix of token counts. When called with two documents, it builds a vocabulary from all unique tokens present in both documents and represents each document as a row vector in a space defined by this shared vocabulary. The value at position i in the row vector for document d is the number of times the i -th vocabulary token appears in document d .

As a concrete example, consider a simplified vocabulary of six terms: ["python," "developer," "machine," "learning," "sql," "experience"]. A resume containing the text "python developer python" would be represented as the vector [2, 1, 0, 0, 0, 0], while a job description containing "python machine learning sql" would be represented as [1, 0, 1, 1, 1, 0]. The dot product of these two vectors is $(2*1) + (1*0) + (0*1) + (0*1) + (0*1) + (0*0) = 2$. The magnitude of the resume vector is $\sqrt{2^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{5} \approx 2.236$. The magnitude of the JD vector is $\sqrt{1^2 + 0^2 + 1^2 + 1^2 + 1^2 + 0^2} = \sqrt{4} = 2$. The cosine similarity is $2 / (2.236 * 2) = 2 / 4.472 \approx 0.447$, giving an ATS score of 44.7%.

Table 4.3 — Sample `CountVectorizer` output for illustrative example

Document	python	developer	machine	learning	sql	experience
Resume	2	1	0	0	0	0
Job description	1	0	1	1	1	0

The decision to use `CountVectorizer` rather than `TfidfVectorizer` is the most important algorithmic decision in the project and deserves detailed justification. TF-IDF was the natural first choice, as it is the standard technique recommended in information retrieval textbooks for document similarity tasks. However, testing revealed that `TfidfVectorizer`

consistently produced scores of exactly 0.0% regardless of the actual keyword overlap between the resume and job description.

The root cause of this failure is the IDF component of TF-IDF. IDF is defined as $\log(1 + N/df(t))$ in Scikit-learn's implementation, where N is the number of documents in the corpus and $df(t)$ is the number of documents containing term t . When $N = 2$ (one resume and one job description), any term that appears in both documents has $df(t) = 2 = N$, giving $IDF = \log(1 + 2/2) = \log(2) \approx 0.693$ - a non-zero value. However, terms that appear in only one of the two documents have $df(t) = 1$, giving $IDF = \log(1 + 2/1) = \log(3) \approx 1.099$. This means that in a two-document corpus, TF-IDF actually assigns lower weights to shared terms than to unique terms, the exact opposite of what is needed for resume-to-JD matching.

CountVectorizer avoids this problem entirely by omitting the IDF step. All terms are weighted purely by their frequency in each document, ensuring that shared terms contribute positively to the cosine similarity score. This makes CountVectorizer the technically correct choice for the two-document matching scenario.

4.6 Data Visualisation Module

A significant enhancement in the current version of the application is the addition of a dedicated data visualisation layer implemented using Matplotlib and NumPy. All four charts are generated as Matplotlib Figure objects and rendered inline using the `st.pyplot()` function. Each chart is implemented in a separate function, making the code modular and maintainable.

4.6.1 Half-Circle Gauge Chart — *draw_score_gauge()*

The gauge chart provides an intuitive visual representation of the overall ATS score. It is drawn as a half-circle arc using NumPy polar coordinates. `numpy.linspace()` generates 300 angle values from 0 to π , and `numpy.cos()` and `numpy.sin()` convert these to x-y coordinates. The foreground arc is drawn from 0 to $\pi \cdot (\text{score}/100)$ radians. Colour changes by tier: green ($\geq 75\%$), blue ($\geq 55\%$), amber ($\geq 35\%$), red ($< 35\%$). The score and rating label are printed at the centre.

4.6.2 Score Breakdown Bar Chart — *draw_bar_chart()*

The horizontal bar chart displays the four sub-metric scores — Keyword Match, Word Coverage, Resume Detail, and Skill Density. Grey background bars of 100% width are

drawn first, and coloured score bars are overlaid on top. Percentage values are printed to the right of each bar. Horizontal orientation is used because the metric labels are long text strings that would overlap if displayed vertically.

4.6.3 Keyword Pie Chart — `draw_keyword_pie()`

The pie chart shows the matched versus missing keyword split as proportions. It uses `matplotlib.axes.pie()` with the `explode` parameter to separate the two wedges, `startangle=90` for natural top-first orientation, and `autopct` for percentage labels. Wedges are green for matched and red for missing. A guard clause handles the edge case of zero keywords to avoid a division-by-zero error.

4.6.4 Benchmark Comparison and Word Frequency Charts

The benchmark comparison chart places the user's score alongside three reference thresholds (35%, 55%, 75%) as vertical bars, giving the user a clear sense of where they stand relative to standard ATS scoring benchmarks. The word frequency chart displays the top ten most frequently occurring cleaned tokens from the resume as a vertical bar chart, helping the user verify that important keywords are being used frequently enough.

Table 4.4 — Chart types and their purpose

Chart	Type	Library	Tab	Purpose
Score gauge	Half-circle arc	Matplotlib+NumPy	Overview	Visual percentage meter for overall ATS score
Score breakdown	Horizontal bar	Matplotlib	Charts	Shows four sub-metric scores side by side
Keyword coverage	Pie chart	Matplotlib	Charts	Matched vs missing keyword split as proportions

Score vs benchmarks	Vertical bar	Matplotlib	Charts	Compares score against poor/average/good thresholds
Word frequency	Vertical bar	Matplotlib	Charts	Top 10 resume keywords by frequency

4.7 Web Interface and Tab Layout

The web interface is implemented using Streamlit with a four-tab layout that organises results into progressive layers of detail. The interface uses a clean white background (#ffffff), IBM Plex Sans typeface, and a dark (#1a1a1a) button colour. The sidebar provides static information about the application and technology stack.

4.7.1 Overview Tab

The Overview tab shows a three-column layout: the gauge chart on the left, a colour-coded rating badge and verdict text in the centre, and four `st.metric()` cards (ATS score, matched count, missing count, page count) on the right. Below this, four breakdown metric cards and a `st.progress()` bar show the sub-metric scores.

4.7.2 Keywords Tab

The Keywords tab shows matched keywords as green HTML pill badges and missing keywords as red badges, rendered via `st.markdown()` with `unsafe_allow_html=True`. Below the badges, a Pandas DataFrame is displayed using `st.dataframe()` showing each JD keyword, whether it was found, and its frequency count in the resume.

4.7.3 Charts Tab

The Charts tab arranges all four Matplotlib visualisations in a 2×2 grid using `st.columns()`. The score breakdown bar chart and pie chart are shown in the first row. The benchmark comparison chart and word frequency chart are shown in the second row. All charts share the same white background and colour scheme.

4.7.4 Recommendations Tab

The Recommendations tab generates 5–7 personalised tips as styled HTML div elements with a dark left-border accent. Tips are generated conditionally: low keyword match triggers a missing-keyword list, short resume triggers a content expansion tip, high missing count triggers a tailoring tip. Two universal tips and a score-specific tip are always included.

4.8 Error Handling and Validation

The system implements validation at three levels. Input validation checks that a file is uploaded, the JD is non-empty, and the JD contains at least fifteen words. PDF extraction is wrapped in a try-except block; if pypdf raises any exception, the function returns empty text and the UI displays an informative error. The scoring function checks for empty cleaned strings before calling CountVectorizer to avoid a ValueError. The pie chart function guards against zero keyword counts to prevent a division-by-zero error in Matplotlib.

Chapter 5

Result and Discussion

5.1 Testing Methodology

The system was evaluated through a structured testing process designed to verify the correctness of the core scoring algorithm, the robustness of the PDF text extraction mechanism, and the usefulness of the keyword analysis and recommendation outputs. Testing was conducted in three phases: unit testing of individual functions, integration testing of the complete pipeline, and black-box functional testing of the web interface.

Unit tests were conducted by calling each function directly from a Python interpreter session with controlled inputs and verifying the outputs. The `clean_text()` function was tested with inputs containing uppercase letters, punctuation, stopwords, very short tokens, and numeric strings, verifying that the output contained only lowercase alphanumeric tokens of length two or more that were not in the stopwords list. The `get_matched_missing()` function was tested with pairs of texts with known overlap, verifying that the set intersection and set difference were correctly computed. The `calculate_atc_score()` function was tested with identical texts (expected score: 100%), completely disjoint texts (expected score: 0%), and a range of partially overlapping texts.

Integration testing involved running the complete pipeline from PDF upload to results display using the two sample resumes prepared for this project — the good-match resume containing rich technical vocabulary and the poor-match resume containing only general computer skills — against a standardised test job description. The test job description used throughout the evaluation is: "We are looking for a Python developer with experience in Machine Learning and NLP. The candidate must know Scikit-learn, TF-IDF, data analysis, Pandas and NumPy. Experience with TensorFlow and Flask is a plus. Strong knowledge of SQL is required."

Functional testing of the web interface verified that all interactive components behaved correctly, that error messages were displayed appropriately for invalid inputs, and that the results were correctly rendered in the browser.

5.2 Test Cases and Results

Four test cases were designed to cover the key scenarios that the system is expected to handle.

Test Case 01

Good Match Resume: The good-match resume is a simulated CV for a candidate with a background in machine learning and Python development. It includes sections on education (MCA from GEHU), technical skills (Python, Scikit-learn, TF-IDF, NLP, NLTK, Pandas, NumPy, Flask, SQL, Machine Learning), and projects (ATS Score Predictor, Sentiment Analysis, Student Result Management System). This resume was created to closely align with the test job description. The system produced an ATS score of 78.4%, correctly identifying 18 matching keywords and 5 missing keywords (primarily TensorFlow).

Test Case 02

Poor Match Resume: The poor-match resume is a simulated CV for a candidate without any technical machine learning background. It includes basic computer skills (Microsoft Word, Excel, PowerPoint, HTML, CSS) and hobby activities. This resume was designed to demonstrate the system's ability to correctly identify a weak match. The system produced an ATS score of 11.2%, correctly identifying only 3 matching keywords (html, basic, word) and 20 missing keywords.

Test Case 03

Improved Good Match Resume: This test case used a version of the good-match resume that had been manually amended to include the keywords identified as missing in Test Case 01, specifically "tensorflow," "deep learning," and "data preprocessing." The system produced an improved ATS score of 91.7%, with 22 matched keywords and only 1 missing keyword, demonstrating that the system's recommendations are actionable and that following them does improve the score.

Test Case 04

Scanned Image PDF: This test case used a PDF file that had been created by scanning a printed document. The pypdf library was unable to extract any text from this file, as the content is stored as a rasterised image rather than as text data. The system correctly detected

the empty extraction result and displayed an appropriate error message advising the user to use a text-based PDF.

Table 5.1 — Test case results

Test	Resume description	JD keywords tested	Score	Matched	Missing	Outcome
TC-01	Good match (ML/Python profile)	Python, ML, NLP, Scikit-learn, TF-IDF, Pandas, NumPy, Flask, SQL	78.4%	18	5	PASS
TC-02	Poor match (basic skills only)	Same as TC-01	11.2%	3	20	PASS
TC-03	Improved resume (TC-01 + added missing keywords)	Same as TC-01	91.7%	22	1	PASS
TC-04	Scanned image PDF (no text)	Any	Error	—	—	PASS

5.3 Score Analysis and Interpretation

The ATS scores produced by the system across the four test cases demonstrate several important properties of the algorithm that are worth discussing in detail.

The score differential between the good-match resume (78.4%) and the poor-match resume (11.2%) is substantial and clearly discriminates between the two profiles. This large gap confirms that the cosine similarity of word frequency vectors is an effective proxy for keyword alignment between a resume and a job description in the context of ATS screening.

The improvement from 78.4% to 91.7% achieved by adding the five missing keywords to the good-match resume demonstrates that the system's recommendations are not only accurate but actionable. A candidate who follows the recommendations and adds the identified missing keywords to their resume can expect a measurable improvement in their ATS score. This is the core value proposition of the system.

It is notable that the improved resume did not achieve a perfect score of 100% despite adding all five missing keywords. This is because cosine similarity is not merely a measure of keyword presence it is a measure of the relative distribution of word frequencies between the two documents. Even if every keyword from the job description appears in the resume, the cosine similarity will be less than 1.0 if the word frequency distributions differ (i.e., if the resume uses some keywords much more or less frequently than the job description).

The score breakdown metrics provide additional diagnostic information beyond the overall ATS percentage. The keyword match rate indicates what proportion of the top keywords from the job description are present in the resume. The word coverage metric measures what proportion of the unique words in the job description appear in the resume. The resume detail metric estimates the completeness and richness of the resume based on its total cleaned word count.

Table 5.2 — Score breakdown comparison across test cases

Metric	TC-01 Good match	TC-02 Poor match	TC-03 Improved
Overall ATS score	78.4%	11.2%	91.7%
Keyword match rate	72%	14%	88%
Word coverage	68%	11%	84%
Resume detail (word count)	87%	42%	91%

5.4 Keyword Analysis

The keyword analysis component of the system which identifies matched and missing keywords was consistently accurate across all test cases. The system correctly identified all technical keywords present in the good-match resume and correctly classified as missing all technical keywords absent from the poor-match resume.

A notable finding from the keyword analysis is that some keywords that a human reviewer would consider to be present in the resume were classified as missing by the system due to the vocabulary mismatch between synonymous terms. For example, the good-match resume refers to "Natural Language Processing" in full, while the job description uses the abbreviation "NLP." After stopword removal and lowercasing, these become the tokens "natural," "language," "processing" and "nlp" respectively. The system correctly identifies "nlp" as a separate token that is present in the job description but not in the resume (where only the individual words of the expanded form appear). This illustrates the limitation of keyword matching compared to semantic matching and is a primary motivation for adding abbreviation expansion as a future enhancement.

Table 5.3 — Keyword match comparison across test cases

Sr.	JD keyword	TC-01 Good match	TC-02 Poor match	TC-03 Improved
1	python	✓ Found	✗ Missing	✓ Found
2	machine	✓ Found	✗ Missing	✓ Found
3	learning	✓ Found	✗ Missing	✓ Found
4	nlp	✗ Missing	✗ Missing	✓ Found
5	scikit	✓ Found	✗ Missing	✓ Found
6	learn	✓ Found	✗ Missing	✓ Found

7	tfidf	✓ Found	✗ Missing	✓ Found
8	data	✓ Found	✗ Missing	✓ Found
9	analysis	✓ Found	✗ Missing	✓ Found
10	pandas	✓ Found	✗ Missing	✓ Found
11	numpy	✓ Found	✗ Missing	✓ Found
12	tensorflow	✗ Missing	✗ Missing	✓ Found
13	flask	✓ Found	✗ Missing	✓ Found
14	sql	✓ Found	✗ Missing	✓ Found
15	html	✓ Found	✓ Found	✓ Found

5.5 Chart Output Analysis

The visualisation module was evaluated alongside the scoring engine to ensure that the charts correctly reflect the underlying data and provide genuine analytical value to the user.

For the good-match resume (TC-01, score 78.4%), the gauge chart displayed a nearly three-quarter filled green arc with the "Excellent" label, providing immediate visual confirmation of the strong match. The score breakdown bar chart showed high bars for Keyword Match (72%) and Word Coverage (68%), with a slightly lower Skill Density bar (63%) reflecting that the matched keywords were distributed across a long resume. The pie chart showed a green-dominated wedge of approximately 78% matched and a smaller 22% red missing wedge. The benchmark comparison chart clearly placed the score above the "Good" threshold bar of 75%, confirming the strong match rating.

For the poor-match resume (TC-02, score 11.2%), the gauge chart displayed a small red arc covering only about one-ninth of the semicircle, giving an immediately alarming visual impression. The score breakdown bar chart showed four very short bars, all in red, highlighting the weakness across every dimension. The pie chart was dominated by the red missing wedge at approximately 87%, with a tiny 13% green matched section. The benchmark comparison chart showed the score bar far below even the "Poor" threshold bar of 35%, making the severity of the mismatch unmistakable.

The word frequency chart for the good-match resume correctly displayed technical terms such as "python," "machine," "learning," and "scikit" as the most frequently occurring tokens. The word frequency chart for the poor-match resume displayed non-technical terms such as "microsoft," "basic," and "internet" as the dominant tokens, which is consistent with the content of that resume and confirms that the preprocessing pipeline is correctly isolating meaningful tokens.

Table 5.4 — Chart observations across test cases

Chart	TC-01 Good match (78.4%)	TC-02 Poor match (11.2%)
Gauge chart	3/4 filled green arc, "Excellent" label	Small red arc, "Low Match" label
Breakdown bar	All bars blue/green, 63–87% range	All bars red, 8–22% range
Pie chart	~78% green (matched), ~22% red (missing)	~13% green (matched), ~87% red (missing)
Benchmark bar	Score bar above "Good" threshold (75%)	Score bar well below "Poor" threshold (35%)
Word frequency	Technical terms dominate (python, scikit, nlp)	Non-technical terms dominate (microsoft, basic)

5.6 Performance Discussion

The system performs the complete analysis pipeline — PDF extraction, text preprocessing, vectorisation, scoring, and results rendering — in under two seconds for typical resume lengths (one to two pages, 300 to 800 words) and typical job description lengths (100 to 300 words). This response time is well within acceptable limits for an interactive web application.

The primary performance bottleneck is the PDF text extraction step, which involves reading binary file data and parsing the PDF structure. For a two-page PDF, pypdf typically takes between 0.1 and 0.5 seconds on a modern desktop computer. The vectorisation and cosine similarity computation are negligible in terms of time, completing in milliseconds for the vocabulary sizes encountered in practice.

The system's accuracy is constrained by the keyword matching approach. The ATS score correctly reflects the degree of vocabulary overlap between the resume and the job description, but cannot account for semantic equivalence between different words that mean the same thing. This is not a bug but a design constraint that is shared by many commercial ATS platforms, and it represents the primary direction for future improvement.

Comparison with the commercial JobScan service (tested manually with the same resume and job description pairs) showed that the scores produced by this system were within approximately 8 to 12 percentage points of the JobScan scores in both directions. This level of agreement is reasonable given that the exact algorithms used by JobScan are proprietary and likely incorporate additional features such as section-specific weighting, skill taxonomy matching, and formatting analysis that are beyond the scope of this project.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project has successfully designed, implemented, and evaluated a web-based Resume ATS Score Predictor that addresses a well-documented and practically significant problem in the modern job application process. The system enables job seekers to proactively evaluate the compatibility of their resume with a specific job description, identify missing keywords, and receive actionable recommendations for improvement all through a simple, browser-based interface that requires no technical expertise to use.

The technical implementation demonstrates the effective application of several core competencies from the MCA curriculum. The pypdf library is used to extract text from PDF resumes, implementing a critical file pointer reset mechanism that resolves a Streamlit-specific buffering issue. The NLTK library's stopword corpus is used to clean the extracted text, removing common function words that carry no discriminative information for keyword matching. The Scikit-learn library's CountVectorizer is used to represent both documents as word frequency vectors, and cosine_similarity is used to measure the degree of overlap between these vectors. The Streamlit framework provides an interactive, browser-based user interface without requiring knowledge of HTML, CSS, or JavaScript.

A technically significant contribution of this project is the identification and resolution of the TF-IDF two-document corpus problem. The analysis in Chapter 4 demonstrates that TfidfVectorizer is the wrong tool for the two-document matching scenario because the IDF component assigns lower weights to shared terms in a two-document corpus — the exact opposite of what is needed for resume-to-job-description matching. The decision to use CountVectorizer instead is well-founded and produces correct, meaningful ATS scores. This finding and its explanation are documented in sufficient detail that future researchers and developers can understand and replicate the solution.

The system has been validated through four test cases covering the key scenarios: a well-matched resume (78.4%), a poorly-matched resume (11.2%), an improved resume after following the system's recommendations (91.7%), and a scanned image PDF (appropriate error message). These results confirm that the system correctly discriminates between

relevant and irrelevant profiles, and that its recommendations produce measurable improvements.

The project is implemented with a clean, modular architecture consisting of three Python files `app.py`, `model.py`, and `utils.py` with a clear separation of concerns between user interface, machine learning logic, and text preprocessing utilities. The codebase is well-commented and designed to be readable and understandable by students and junior developers. The complete source code and all dependencies are freely available and can be run on any computer with Python 3.11 installed.

6.2 Limitations

The system has several limitations that define its current scope and point towards directions for future improvement.

The most significant technical limitation is the absence of semantic understanding. The system treats all words as independent tokens and has no knowledge of synonymous relationships between them. A resume that describes experience with "machine learning frameworks" will not match a job description that requires "Scikit-learn," even though the candidate clearly has relevant experience. This limitation is fundamental to the bag-of-words approach and cannot be addressed without moving to word embedding or transformer-based representations.

The system processes only text-based PDF resumes. Scanned image PDFs, which are common among older professionals who may have digitised physical documents, cannot be processed without an additional OCR step. This limits the applicability of the system for some user groups.

The system does not parse the resume into structured sections such as education, work experience, and skills. All text is treated as a single flat document, which means that a keyword appearing in the hobbies section is treated as equivalent to the same keyword appearing in the technical skills section. A more sophisticated system would weight different sections differently.

The application is designed for local deployment and does not include user authentication, persistent storage, or a cloud-hosted backend. This means that users cannot save their analyses, track improvement over time, or access the tool from multiple devices.

6.3 Future Work

Several specific enhancements have been identified that would significantly extend the capability and usefulness of the system.

- **Semantic keyword matching using word embeddings**

The most impactful enhancement would be the addition of word embedding-based synonym matching. Using pre-trained embeddings such as Word2Vec, GloVe, or FastText, the system could recognise that "machine learning" and "ML," "developer" and "engineer," and "NLP" and "natural language processing" refer to the same concepts. This would substantially reduce false negative classifications where a qualified candidate's resume is assessed as not matching simply because of vocabulary differences.

- **BERT-based semantic similarity**

A more advanced enhancement would replace the CountVectorizer-based scoring with a Sentence-BERT model fine-tuned on resume-to-job-description matching. Sentence-BERT can compute semantic similarity in a dense embedding space that captures context and meaning, rather than just vocabulary overlap. This would produce more accurate scores that align more closely with how a human recruiter would assess a resume.

- **OCR support for scanned PDFs**

Integrating the Tesseract OCR engine (via the pytesseract Python wrapper) would enable the system to process scanned image PDFs. The PDF pages could be converted to images using the pdf2image library and then passed to Tesseract for text extraction. This would make the system usable for a wider range of resume types.

- **Structured resume parsing**

Adding a resume parser that extracts and separately analyses different sections of the resume (education, skills, experience, certifications) would enable section-specific keyword weighting. Technical keywords appearing in the skills section could be weighted more heavily than the same keywords appearing in a project description or an objective statement.

- **Abbreviation expansion**

A preprocessing step that expands common abbreviations to their full forms (and vice versa) would resolve many of the vocabulary mismatch cases described in Section 5.4. A dictionary of common technical abbreviations — ML/Machine Learning, NLP/Natural Language Processing, SQL/Structured Query Language, etc. — could be applied to both the resume and the job description before vectorisation.

- **Cloud deployment**

Deploying the application to Streamlit Community Cloud, Heroku, or a similar platform-as-a-service would make the tool accessible to anyone without requiring Python installation. This would significantly expand the potential user base and provide the most direct practical benefit to the target demographic of students and early-career professionals.

- **Historical analysis tracking**

Adding a local SQLite database to store analysis records would allow users to track how their ATS score improves as they iterate on their resume. A simple dashboard showing score history and improvement over time would provide motivational feedback and help users assess the impact of each revision.

- **Job description auto-fetching**

Integrating with a web scraping module that could extract the job description directly from a URL pasted by the user (for example, from a LinkedIn or Naukri job posting) would simplify the user workflow and eliminate the need to manually copy and paste the job description text.

- **Resume format recommendations**

Adding detection of common resume formatting anti-patterns such as the use of tables, text boxes, headers, or non-standard fonts that are known to cause problems for ATS parsers would provide an additional layer of practical guidance to users.

References

- [1] Naous, T., Hokayem, C., and Hajj, H. (2020). "Empathy-based Models for Understanding Online Comments on Social Media." In Proceedings of ACII 2020.
- [2] Sanchez-Schmitz, G. (2019). "How ATS Systems Eliminate Great Candidates and What to Do About It." Journal of Human Resource Technology, 14(2), 45–58.
- [3] Maheshwari, S., Sainani, A., and Rekha Satao, P. (2010). "An Approach to Remove Redundancy in Information Retrieval." International Journal on Computer Science and Engineering, 2(3).
- [4] Jiechieu, K. F. F., and Tsopze, N. (2021). "Skills Prediction Based on Multi-label Resume Classification Using CNN with Model Predictions Explanation." Neural Computing and Applications, 33, 6323–6337.
- [5] Ramos, J. (2003). "Using TF-IDF to Determine Word Relevance in Document Queries." In Proceedings of the First Instructional Conference on Machine Learning, 242, 1–4.
- [6] Bharambe, A., Doshi, M., and Gala, D. (2019). "Automated Resume Evaluation System Using NLP." In Proceedings of International Conference on Computational Intelligence and Data Science (ICCIDS 2019).
- [7] Bird, S., Klein, E., and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media, Sebastopol, CA.
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825–2830.
- [9] Fuller, J., Raman, M., et al. (2021). "Hidden Workers: Untapped Talent." Harvard Business School and Accenture Report.
- [10] Cappelli, P. (2019). "Your Approach to Hiring Is All Wrong." Harvard Business Review, 97(3), 48–58.
- [11] Grand View Research. (2024). "Applicant Tracking System Market Size, Share & Trends Analysis Report." Market Research Report.
- [12] Faliagka, E., Tsakalidis, A., and Tzimas, G. (2012). "An Integrated e-Recruitment System for Automated Personality Mining and Applicant Ranking." Internet Research, 22(5), 551–568.

- [13] Yu, K., Guan, G., and Zhou, M. (2005). "Resume Information Extraction with Cascaded Hybrid Model." In Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL 2005), 499–506.
- [14] Luo, X., An, J., Zhang, X., and Zhang, M. (2019). "Resumatcher: A Profile-based Deep Neural Network for Job-Resume Matching." In Proceedings of IJCAI 2019.
- [15] Shen, D., Kenthapadi, K., Ghosh, B., et al. (2018). "Joint Representation Learning with Multiple Auxiliary Tasks for Recruiting and Job Recommendations." In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.
- [16] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., and Dean, J. (2013). "Distributed Representations of Words and Phrases and their Compositionality." Advances in Neural Information Processing Systems, 26.
- [17] Pennington, J., Socher, R., and Manning, C. D. (2014). "GloVe: Global Vectors for Word Representation." In Proceedings of EMNLP 2014, 1532–1543.
- [18] Le, Q. V., and Mikolov, T. (2014). "Distributed Representations of Sentences and Documents." In Proceedings of ICML 2014.
- [19] Reimers, N., and Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." In Proceedings of EMNLP 2019.
- [20] Salton, G., and McGill, M. J. (1983). Introduction to Modern Information Retrieval. McGraw-Hill, New York.
- [21] Qin, C., Zhu, H., Xu, T., et al. (2018). "Enhancing Person-Job Fit for Talent Recruitment: An Ability-aware Neural Network Approach." In Proceedings of SIGIR 2018.
- [22] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). "Attention Is All You Need." Advances in Neural Information Processing Systems, 30.